AIML Project

VIT Bhopal SGPA and CGPA Calculator

Name- Pratham Yadav

REG No. –25BAI11032

Course title- Fundamental of AIML

1. Executive Summary

This project is a Python command-line application that helps students calculate SGPA and CGPA using the VIT Bhopal 10-point grading system. The calculator supports three practical use cases: semester-wise SGPA calculation from subject grades, CGPA calculation from semester SGPAs, and CGPA calculation directly from subject-level data across multiple semesters.

The main purpose of the project is to remove the confusion and errors that often happen during manual grade calculations. Instead of asking students to repeatedly apply weighted-average formulas by hand, the program guides them through the process, validates their input, and returns the final result clearly.

Although the project is technically simple, it addresses a real student need. It also demonstrates core programming skills such as modular design, input validation, use of dictionaries, loops, conditionals, and function-based decomposition.

2. Problem Chosen

The problem chosen for this project is the accurate calculation of SGPA and CGPA for students following the VIT Bhopal grading pattern.

Students often want to know:

- their SGPA for a single semester,
- their CGPA across multiple semesters,
- or their overall performance directly from subject-wise grades.

In practice, these calculations are not simple averages. They are weighted averages based on course credits. Because of that, manual calculation becomes repetitive and error-prone, especially when a student has many subjects or wants to combine several semesters.

The core problem can be stated as:

How can we build a simple and reliable calculator that computes SGPA and CGPA correctly using the VIT Bhopal 10-point grading system while also reducing mistakes caused by invalid input?

3. Why This Problem Matters

This problem matters because SGPA and CGPA are more than just academic numbers. Students use them to:

- track academic progress,
- estimate eligibility for internships and placements,
- check scholarship requirements,
- understand how each semester affects long-term performance,
- and plan improvement strategies for future semesters.

Even small mistakes in calculation can create confusion. For example, students may average grades directly without considering credits, or they may mix semester averages incorrectly. A tool that automates the weighted calculation gives faster and more dependable results.

This project also matters from a software perspective because it takes a real-world student problem and turns it into a practical program. It shows how programming can be used not only for large systems, but also for small tools that solve everyday problems effectively.

4. Project Objectives

The main objectives of the project were:

- to build a working SGPA and CGPA calculator in Python,
- to use the VIT Bhopal 10-point grading system,
- to support multiple methods of CGPA calculation,
- to validate user input so the program remains reliable,
- to keep the design simple enough for command-line use,
- and to write the solution in a modular, readable way.

5. Scope of the Project

The current version of the project includes:

- SGPA calculation from subject credits and grades,
- CGPA calculation from semester SGPA values and semester credits,
- CGPA calculation from all course grades across multiple semesters,
- grade-to-point conversion through a fixed mapping,
- and input validation for numbers, ranges, and valid grades.

The project does not currently include:

- a graphical user interface,
- data storage for past semesters,
- export to PDF or formatted report files,
- authentication or multi-user support,
- or support for grading systems other than the VIT-style mapping currently coded into the program.

6. Understanding the Problem Domain

The project is based on a standard academic rule: SGPA and CGPA are weighted averages, not plain arithmetic means.

The code uses the following grade mapping:

Grade	Grade Point
S	10
A	9
B	8
C	7
D	6
E	5
F	0
N	0

The formulas implemented are:

SGPA

SGPA = Sum(Credit x Grade Point) / Sum(Credits)

CGPA from semester SGPAs

CGPA = Sum(Semester SGPA x Semester Credits) / Sum(Semester Credits)

CGPA from course grades

CGPA = Sum(Credit x Grade Point for all subjects) / Sum(All Credits)

The program also shows an approximate percentage using:

Percentage = CGPA x 10

This is useful for rough interpretation, although it is clearly not guaranteed to match official university conversion rules exactly.

7. Approach to Solving the Problem

The approach was to build a menu-driven terminal application that walks the user through one calculation path at a time. Instead of trying to solve every possible grading scenario, the project focuses on the most common needs a student has.

The overall design breaks the problem into four parts:

7.1 Representing the grading system

The grading system is represented using
the GRADE_POINTS dictionary. This is a good fit because it allows fast lookup from a letter grade such as A or S to the corresponding numeric value needed for computation.

7.2 Validating user input

Several helper functions were written to reduce mistakes and prevent crashes:

- get_positive_int(prompt)
- get_positive_float(prompt)
- get_float_in_range(prompt, minimum, maximum)
- get_grade(prompt)

These functions centralize validation logic so the rest of the calculator can focus on the actual formula instead of repeated error handling.

7.3 Separating the calculator modes

The three major use cases are split into separate functions:

- calculate_sgpa()
- calculate_cgpa_from_semesters()
- calculate_cgpa_from_courses()

This decision improves clarity. Each function has a single responsibility and can be understood independently.

7.4 Using a small control layer

The main() function acts as the control layer. It prints the title, explains the grading table, shows the menu, and routes the user to the correct calculation function based on the selected option.

This makes the program structure easy to follow:

1. show information,
2. take user choice,
3. collect validated inputs,
4. compute weighted result,
5. display output.

8. How the Program Works

When the application starts, it displays a heading and the grade table. Then the user chooses one of three options.

Option 1: SGPA from subject grades

The program asks for the number of subjects in one semester. For each subject, it accepts:

- subject name,
- credit value,
- and grade.

It then multiplies each subject's credit by the grade point, sums all weighted points, divides by total credits, and prints the SGPA.

Option 2: CGPA from semester SGPAs

The program asks for the number of completed semesters. For each semester, it accepts:

- semester SGPA,
- and total credits for that semester.

It then computes the weighted average of the semester SGPAs instead of incorrectly taking a plain average.

Option 3: CGPA from all course grades

The program asks for the number of semesters to include. For each semester, it asks for the number of subjects and then collects subject-level data. This gives a more detailed CGPA calculation directly from raw academic records.

SCREENSHOOTS

CODE:

```python
GRADE_POINTS = {
    "S": 10,
    "A": 9,
    "B": 8,
    "C": 7,
    "D": 6,
    "E": 5,
    "F": 0,
    "N": 0,
}

def get_positive_int(prompt):
    while True:
        value = input(prompt).strip()
        try:
            number = int(value)
            if number > 0:
                return number
        except ValueError:
            pass
        print("Please enter a valid positive whole number.")


def get_positive_float(prompt):
    while True:
        value = input(prompt).strip()
        try:
            number = float(value)
            if number > 0:
                return number
        except ValueError:
            pass
        print("Please enter a valid positive number.")


def get_float_in_range(prompt, minimum, maximum):
    while True:
        value = input(prompt).strip()
```

MENU:

```
A = 9
B = 8
C = 7
D = 6
E = 5
F = 0
N = 0

Choose an option:
1. Calculate SGPA from subject grades
2. Calculate CGPA from semester SGPAs
3. Calculate CGPA directly from all course grades
Enter 1, 2, or 3:
```

RESULT:

```
SGPA CALCULATOR
Enter number of subjects in the semester: 1

Subject 1
Subject name: MATH
Credits for MATH: 4
Grade for MATH: A

RESULT
Total credits: 4.00
Total grade points: 36.00
SGPA: 9.00
PS C:\Users\Pratham>
```

9. Key Design Decisions

Several design decisions shaped the final program.

9.1 Choosing a command-line interface

The project uses a terminal-based interface instead of a GUI. This keeps the focus on logic, avoids unnecessary complexity, and makes the program easy to run on any system with Python installed.

This was a practical decision because the main goal was accurate calculation, not interface design.

9.2 Using functions instead of one long script

Breaking the program into smaller functions improves readability and maintainability. It also makes the code easier to debug because each function solves one clear part of the problem.

9.3 Supporting three calculation paths

This is one of the strongest decisions in the project. Different students have different data available. Some know their subject-level grades, while others only know semester SGPAs. Supporting all three modes makes the tool more useful in real situations.

9.4 Enforcing validation before calculation

The project does not assume perfect input from the user. Invalid grades, negative credits, and out-of-range SGPA values are all rejected and requested again. This design choice directly improves correctness and user trust.

9.5 Accepting decimal credits

Credits are handled as floating-point values instead of integers only. This makes the tool more flexible and realistic.

9.6 Displaying intermediate totals

The output includes total credits and weighted

10. Challenges Faced

Even for a small project, there were important design and implementation challenges.

10.1 Preventing incorrect input

The biggest challenge was handling real user input safely. Users may enter:

- letters instead of numbers,
- negative values,
- zero where positive values are required,
- unsupported grades,
- or blank inputs.

If input is not validated carefully, the calculator can crash or produce meaningless results.

10.2 Keeping the flow simple across multiple modes

Because the program supports three separate workflows, there was a risk of making the interface confusing. The prompts needed to stay straightforward while still collecting enough information for each calculation.

10.3 Avoiding repeated logic

Validation and weighted-average operations appear in multiple places conceptually. Without planning, the code could easily have become repetitive and harder to maintain.

10.4 Balancing simplicity with usefulness

The project had to remain small and beginner-friendly while still solving a genuine academic task well. It was important not to overload the first version with extra features that would distract from the core calculator.

11. How Those Challenges Were Addressed

The project solves these challenges in a practical way.

- Input errors are handled through reusable validation functions that loop until valid input is entered.
- Multiple workflows are managed by placing each calculation path in its own dedicated function.
- Shared concepts like grade mapping are centralized instead of being duplicated.
- The interface remains simple by using a single menu and direct prompts instead of complicated navigation.

This is a good example of how even a small program benefits from separating concerns and thinking about the user experience.

12. Code Quality Observations

From reading the code, several positive implementation choices stand out:

- variable names are clear and descriptive,
- the program flow is easy to trace,
- validation logic is consistently reused,
- calculations are straightforward and readable,
- and the functions are small enough to understand quickly.

There are also some natural limitations in the current implementation:

- the program runs for one calculation and then exits instead of looping back to the menu,
- there is no persistence of records,
- and there are no automated tests included in the repository.

13. Testing and Verification Strategy

For this type of project, testing should focus on both correctness and input safety.

Useful test scenarios include:

- SGPA calculation with normal subject data,
- CGPA calculation where semester credits are different,
- CGPA calculation from multiple semesters of subject-wise entries,
- invalid grades such as P or X,
- negative or zero credits,
- non-numeric values where numbers are expected,
- and boundary values such as SGPA 0 and 10.

The expected behavior is:

- valid inputs are accepted,
- invalid inputs trigger a retry message,
- weighted formulas are applied correctly,

14. What I Learned From the Project

This project provides learning in both programming and problem-solving.

Technical learning

- how to use dictionaries for real data mapping,
- how to structure a Python program with helper functions,
- how to validate user input safely,
- how to implement weighted-average calculations correctly,
- and how modularity improves readability even in short programs.

Practical learning

- real users make unexpected input mistakes, so validation matters,
- small tools can still solve meaningful problems,
- interface clarity is important even in terminal applications,
- and choosing the right scope is an important design skill.

Personal development as a developer

The project reinforces an important lesson: a useful program does not need to be large or complex. If it solves a real problem accurately and clearly, it already has value. This mindset is especially important for beginner and intermediate projects.

15. Strengths of the Solution

The strongest aspects of the project are:

- it solves a real and relevant student problem,
- it uses the correct weighted logic instead of simple averaging,
- it supports more than one path to CGPA calculation,
- it protects against invalid input,
- it is easy to run with only Python installed,
- and it demonstrates clean beginner-friendly structure.

16. Limitations

The current version still has some limitations:

- it only supports one grading scheme,
- it is limited to command-line interaction,
- it does not save or export academic history,
- it does not provide semester comparison analytics,
- and the percentage conversion is only approximate.

Recognizing these limitations is useful because it shows where the project can evolve next.

17. Future Improvements

There are several realistic ways to extend the project:

- add a loop so users can perform multiple calculations in one run,
- create a graphical interface using Tkinter or another GUI framework,
- save semester data to a file or database,
- export results as text or PDF,
- support different university grading systems,
- add semester-wise performance tracking,
- and include automated test cases for the calculation functions.

These changes would improve both usability and software quality without changing the core purpose of the project.

18. Conclusion

The VIT Bhopal SGPA and CGPA Calculator is a focused and useful Python project built around a real academic need. It shows a clear understanding of the problem, applies the correct weighted formulas, and improves reliability through careful input validation.

More importantly, the project demonstrates thoughtful design choices. The use of a grade dictionary, helper validation functions, and separate calculation modes makes the code organized and practical. While the program can be expanded in future versions, the current implementation already succeeds as both a functional student tool and a strong learning project in Python.

In summary, this project matters because it combines correctness, usability, and simple software design to solve a problem students genuinely face. That combination makes it a meaningful and effective piece of work.

MADE BY- PRATHAM YADAV

REG NO. – 25BAI11032